



**Частное учреждение профессионального образования
«Высшая школа предпринимательства»
(ЧУПО «ВШП»)**

КУРСОВОЙ ПРОЕКТ

«Создание базы данных для сайта по продаже мерча»

Выполнил:
студент 3-го курса специальности
09.02.07 «Информационные системы
и программирование»
Иванькова Варвара Витальевна
подпись: _____

Проверил:
преподаватель дисциплины,
преподаватель ЧУПО «ВШП»,
к.ф.н. Ткачев П.С.
оценка: _____
подпись: _____

Тверь, 2026 г.

ОГЛАВЛЕНИЕ

ОГЛАВЛЕНИЕ.....	2
ВВЕДЕНИЕ.....	3
Глава 1. Анализ предметной области.....	4
Глава 2. Проектирование базы данных.....	7
Глава 3. Реализация базы данных и приложения на Python.....	12
Глава 4. Тестирование и примеры запросов.....	21
ЗАКЛЮЧЕНИЕ.....	29
СПИСОК ЛИТЕРАТУРЫ.....	30
Ссылка на гит.....	31

ВВЕДЕНИЕ

В современном мире электронная коммерция занимает ключевое место в экономике. Продажа мерча (сувенирной продукции, одежды, аксессуаров с символикой) через интернет-магазины становится всё более популярной. Эффективное управление товарными запасами, клиентской базой и заказами требует автоматизированной системы хранения данных.

Данный курсовой проект посвящен разработке базы данных для сайта по продаже мерча. Цель проекта — создать структурированное хранилище информации о товарах, клиентах и заказах, а также разработать прикладное приложение для управления этими данными.

В традиционном (ручном) учёте используются бумажные журналы и таблицы Excel, что приводит к следующим проблемам:

- сложность отслеживания остатков товаров в реальном времени;
- трудоёмкость формирования списка популярных товаров;
- риск потери или ошибки при записи данных о заказах;
- невозможность быстрого получения аналитической информации.

Автоматизированная система должна решить эти задачи.

Задачи проекта:

1. Проанализировать деятельность интернет-магазина мерча и выделить основные сущности.
2. Спроектировать инфологическую и реляционную модель базы данных.
3. Реализовать базу данных в СУБД MySQL.

4. Разработать приложение на Python, реализующее CRUD-операции, оформление заказов и формирование отчётов.
5. Выполнить тестирование и привести примеры SQL-запросов.

Глава 1. Анализ предметной области

Описание предметной области

Интернет-магазин по продаже мерча занимается реализацией товаров с символикой музыкальных групп, брендов, фильмов или игр. Ассортимент включает одежду (футболки, худи, свитшоты), аксессуары (кружки, значки, постеры) и сувенирную продукцию. Клиенты выбирают товары, оформляют заказы, указывают адрес доставки и оплачивают покупки. Менеджер магазина отслеживает статус заказов и управляет товарными остатками.

В настоящее время учёт ведётся в электронных таблицах, что создаёт ряд проблем:

- данные о товарах и заказах хранятся в разных файлах;
- при оформлении заказа невозможно автоматически проверить наличие товара;
- отчёты о продажах формируются вручную и занимают много времени;
- отсутствует единая база клиентов.

Разрабатываемая система должна автоматизировать все эти процессы и обеспечить удобный доступ к информации.

Основные сущности и процессы

В результате анализа деятельности интернет-магазина выделены следующие сущности:

Товар — содержит информацию о наименовании, категории, цене, размере (для одежды) и количестве на складе.

Клиент — содержит данные о покупателе: фамилию, имя, отчество, контактные данные (email, телефон) и адрес доставки.

Заказ — фиксирует факт оформления покупки: клиент, дата оформления, статус заказа, итоговая сумма и адрес доставки.

Состав заказа — связывает заказ и товары, указывая количество и цену каждого товара на момент заказа.

Требования к системе

Функциональные требования:

- Хранение данных о товарах, клиентах, заказах и их составе.
- Выполнение операций CRUD (создание, чтение, обновление, удаление) для каждой сущности.
- Контроль доступности товаров при оформлении заказа.
- Автоматический расчёт итоговой суммы заказа.
- Формирование трёх обязательных отчётов (популярные товары, выручка по категориям, активные клиенты).

- Возможность просмотра остатков товаров.

Нефункциональные требования:

- Простота использования (консольное меню с цифрами).
- Надёжность (целостность данных через внешние ключи).
- Масштабируемость (возможность добавления новых таблиц и полей).
- Безопасность (защита от некорректных данных через ограничения).

Глава 2. Проектирование базы данных

Инфологическая модель (ER-диаграмма)

В курсовом проекте используется нотация «сущность-связь». Выделены четыре сущности:

ТОВАРЫ (Products) — содержит сведения обо всех товарах, включая их характеристики и количество на складе.

КЛИЕНТЫ (Customers) — содержит данные о пользователях интернет-магазина.

ЗАКАЗЫ (Orders) — фиксирует каждый факт оформления заказа клиентом.

СОСТАВ ЗАКАЗА (Order_Items) — содержит перечень товаров в каждом заказе с указанием количества и цены.

Связи:

- Один клиент может оформить много заказов (связь 1 : ∞).
- Один заказ может включать много товаров (связь 1 : ∞).
- Один товар может быть включен во много заказов (связь 1 : ∞).

Таким образом, связь между заказами и товарами является связью "многие ко многим", которая реализуется через промежуточную таблицу "Состав заказа".

Логическая и физическая модели

Логическая модель (таблицы и поля)

Таблица Products (товары)

Поле	Тип данных	Описание
id	целое число	Первичный ключ, автоинкремент
name	текст	Название товара
category	текст	Категория товара
price	число с плавающей точкой	Цена в рублях
size	текст	Размер (для одежды)
stock_quantity	целое число	Количество на складе
created_at	дата/время	Дата добавления товара

Таблица Customers (клиенты)

Поле	Тип данных	Описание
id	целое число	Первичный ключ, автоинкремент
full_name	текст	ФИО клиента
email	текст	Электронная почта
phone	текст	Номер телефона
address	текст	Адрес доставки
registered_at	дата/время	Дата регистрации

Таблица Orders (заказы)

Поле	Тип данных	Описание
id	целое число	Первичный ключ, автоинкремент
customer_id	целое число	Внешний ключ к Customers(id)
order_date	дата/время	Дата оформления заказа
status	текст	Статус заказа
total_amount	число с плавающей точкой	Итоговая сумма заказа
shipping_address	текст	Адрес доставки

Таблица Order_Items (состав заказа)

Поле	Тип данных	Описание
id	целое число	Первичный ключ, автоинкремент
order_id	целое число	Внешний ключ к Orders(id)
product_id	целое число	Внешний ключ к Products(id)
quantity	целое число	Количество товара
price_at_order	число с плавающей точкой	Цена на момент заказа

Физическая модель (реализация в MySQL)

Для реализации базы данных в СУБД MySQL используются следующие типы данных:

Тип в логической модели	Тип в MySQL
-------------------------	-------------

целое число	INT
текст	VARCHAR(200), TEXT
число с плавающей точкой	DECIMAL(10,2)
дата/время	TIMESTAMP
статус	ENUM('новый','оплачен','отправлен','доставлен','отменён')

Все таблицы имеют первичный ключ с автоинкрементом. Внешние ключи обеспечивают ссылочную целостность данных:

- `orders.customer_id` → `customers.id`
- `order_items.order_id` → `orders.id`
- `order_items.product_id` → `products.id`

Нормализация

Проведём проверку на соответствие нормальным формам.

Первая нормальная форма (1НФ): Все атрибуты атомарны, нет повторяющихся групп. В таблице Products нет списка размеров или категорий в одной ячейке — выполнено. В таблице Orders нет списка товаров в одном поле — выполнено.

Вторая нормальная форма (2НФ): Таблицы содержат только атрибуты, зависящие от полного ключа. В таблице Order_Items используется суррогатный ключ id, а все атрибуты зависят от него. Составных ключей нет — 2НФ выполнена.

Третья нормальная форма (3НФ): Нет транзитивных зависимостей. Например, `price_at_order` в `Order_Items` не зависит от других полей, а фиксируется явно (это сделано для сохранения исторической цены). `total_amount` в `Orders` рассчитывается на основе `Order_Items`, но хранится отдельно для удобства — это допустимое нарушение 3НФ, оправданное производительностью.

Таким образом, структура базы данных является нормализованной и исключает избыточность и аномалии обновления. Все таблицы находятся в 3НФ.

Глава 3. Реализация базы данных и приложения на Python

Для реализации базы данных интернет-магазина мерча был выбран ряд инструментов, обеспечивающих эффективное проектирование, создание и тестирование структуры данных.

В качестве системы управления базами данных выбрана MySQL. Данный выбор обусловлен рядом преимуществ. MySQL является одной из наиболее популярных реляционных СУБД, широко применяемых в веб-разработке и корпоративных информационных системах. Она обеспечивает высокую производительность, надежность и поддержку всех необходимых функций для обеспечения целостности данных, включая внешние ключи, ограничения уникальности, транзакции и механизмы блокировок. Кроме того, MySQL распространяется под свободной лицензией, что делает её доступной для использования в учебных целях.

Для проектирования и администрирования базы данных выбрана среда MySQL Workbench. Данный инструмент предоставляет удобный визуальный интерфейс для создания таблиц, установления связей между ними, написания и выполнения SQL-запросов. Особую ценность представляет возможность построения ER-диаграмм, которые наглядно отображают структуру базы данных и взаимосвязи между сущностями.

Это значительно упрощает процесс проектирования и позволяет быстро выявлять ошибки в структуре данных.

В качестве языка запросов используется стандартный диалект SQL, поддерживаемый MySQL. SQL является универсальным языком для работы с реляционными базами данных и позволяет выполнять все необходимые операции: создание и модификацию таблиц, добавление, изменение и удаление данных, а также формирование сложных аналитических запросов.

Таким образом, выбранный набор инструментов полностью соответствует задачам курсового проекта и позволяет реализовать базу данных, удовлетворяющую всем требованиям.

Создание базы данных

Процесс создания базы данных в MySQL Workbench начинается с установления соединения с сервером MySQL и выполнения SQL-команды `CREATE DATABASE`. В рамках данного проекта база данных получает название `merch_shop`, которое отражает её предназначение — хранение информации об интернет-магазине мерча. При создании базы данных задаётся кодировка UTF-8, обеспечивающая корректное хранение текстовой информации на русском языке.

После создания базы данных выполняется переход в неё с помощью команды `USE merch_shop`, после чего создаются все необходимые

таблицы. Все действия выполняются с использованием оператора `IF NOT EXISTS`, что гарантирует корректную работу скрипта при его повторном выполнении — таблицы будут созданы только в том случае, если они ещё не существуют.

Структура базы данных

Разработанная база данных состоит из четырёх таблиц, соответствующих выделенным на этапе проектирования сущностям: `products` (товары), `customers` (клиенты), `orders` (заказы) и `order_items` (состав заказа).

Таблица `products` предназначена для хранения информации о товарах, представленных в интернет-магазине. Она содержит следующие поля: уникальный идентификатор товара (`id`), название товара (`name`), категория (`category`), цена (`price`), размер (`size`), количество на складе (`stock_quantity`) и дата добавления товара в базу данных (`created_at`). Поле `id` является первичным ключом с автоматическим увеличением значения. Поле `name` обязательно для заполнения, так как каждый товар должен иметь название. Поле `price` также обязательно и хранится в формате десятичного числа с двумя знаками после запятой для точного отображения денежных сумм. Поле `stock_quantity` имеет значение по умолчанию 0, что позволяет корректно обрабатывать ситуации отсутствия товара на складе. Поле `size` предназначено для указания размеров одежды,

для товаров, не имеющих размера (например, кружки или постеры), данное поле может оставаться пустым.

Таблица `customers` содержит информацию о клиентах интернет-магазина. Она включает идентификатор клиента (`id`), полное имя (`full_name`), адрес электронной почты (`email`), номер телефона (`phone`), адрес доставки (`address`) и дату регистрации (`registered_at`). Поле `email` имеет ограничение уникальности, что гарантирует отсутствие дублирующихся записей об одном клиенте и позволяет использовать адрес электронной почты в качестве логина для будущей системы авторизации. Поля `full_name` и `phone` являются обязательными для заполнения. Поле `address` хранит основной адрес доставки клиента, который может быть изменён при оформлении конкретного заказа.

Таблица `orders` фиксирует каждый заказ, оформленный клиентом. Структура таблицы включает идентификатор заказа (`id`), идентификатор клиента (`customer_id`), дату оформления заказа (`order_date`), статус заказа (`status`), итоговую сумму (`total_amount`) и адрес доставки (`shipping_address`). Поле `customer_id` является внешним ключом, ссылающимся на поле `id` в таблице `customers`, что обеспечивает ссылочную целостность данных. Поле `status` представляет собой перечисление допустимых значений: 'новый', 'оплачен', 'отправлен', 'доставлен', 'отменён'. Это позволяет стандартизировать данные и исключить ошибки при вводе статусов. Поле `total_amount` хранит итоговую сумму заказа, рассчитанную на основе данных из таблицы

`order_items`. Поле `shipping_address` хранит конкретный адрес доставки для данного заказа, который может отличаться от основного адреса клиента.

Таблица `order_items` является связующей между заказами и товарами и реализует связь "многие ко многим". Она содержит идентификатор записи (`id`), идентификатор заказа (`order_id`), идентификатор товара (`product_id`), количество товара (`quantity`) и цену на момент заказа (`price_at_order`). Поле `order_id` является внешним ключом к таблице `orders` с каскадным удалением записей при удалении заказа. Поле `product_id` является внешним ключом к таблице `products` с ограничением на удаление товара, если он фигурирует в каком-либо заказе. Поле `price_at_order` фиксирует цену товара на момент оформления заказа, что позволяет сохранить историческую информацию о стоимости товаров даже в случае их последующего изменения. Это особенно важно для формирования корректных финансовых отчётов за прошедшие периоды.

Ограничения целостности данных

В разработанной базе данных реализован ряд ограничений, обеспечивающих целостность и корректность хранимой информации.

Первичные ключи (PRIMARY KEY) в каждой таблице гарантируют уникальность каждой записи и обеспечивают быстрый доступ к данным по идентификатору. Во всех четырёх таблицах первичным ключом является поле `id` с автоматическим увеличением значения.

Ограничение уникальности (UNIQUE) на поле `email` в таблице `customers` предотвращает создание дублирующихся записей об одном клиенте и гарантирует, что каждый адрес электронной почты может быть использован только одним пользователем.

Ограничение NOT NULL применяется к полям, которые обязательно должны быть заполнены: названия товаров, цены, ФИО и телефоны клиентов, идентификаторы внешних ключей, цены и количества товаров в заказах.

Внешние ключи (FOREIGN KEY) обеспечивают ссылочную целостность данных. В таблице `orders` внешний ключ `customer_id` ссылается на `customers(id)` с опцией `ON DELETE RESTRICT`, что предотвращает удаление клиента, имеющего заказы. В таблице `order_items` внешний ключ `order_id` ссылается на `orders(id)` с опцией `ON DELETE CASCADE`, что обеспечивает автоматическое удаление состава заказа при удалении самого заказа. Внешний ключ `product_id` ссылается на `products(id)` с опцией `ON DELETE RESTRICT`, что предотвращает удаление товара, если он присутствует в каком-либо заказе.

Все перечисленные ограничения работают на уровне СУБД и автоматически проверяются при выполнении операций добавления, изменения и удаления данных, что исключает возможность появления некорректных или "сиротских" записей.

Схема базы данных

Для наглядного представления структуры разработанной базы данных в среде MySQL Workbench была построена ER-диаграмма (диаграмма "сущность-связь"). Данная диаграмма визуально отображает все таблицы, их поля и связи между ними.

Центральной таблицей в схеме является таблица `orders`, которая связывает клиентов и товары. С одной стороны, таблица `orders` соединена с таблицей `customers` через внешний ключ `customer_id` (связь "один ко многим": один клиент может иметь много заказов). С другой стороны, таблица `orders` соединена с таблицей `order_items` (связь "один ко многим": один заказ может содержать много товаров). Таблица `order_items`, в свою очередь, соединена с таблицей `products` через внешний ключ `product_id` (связь "один ко многим": один товар может входить во много заказов).

Таким образом, схема базы данных полностью отражает бизнес-логику интернет-магазина и позволяет отслеживать все этапы работы с заказами: от добавления товаров до формирования отчётности.

Заполнение базы данных тестовыми данными

Для проверки работоспособности базы данных в неё были внесены тестовые данные. Наполнение таблиц выполнялось с учётом необходимости охвата различных сценариев использования системы.

В таблицу `products` добавлены пять товаров, относящихся к разным категориям: футболки, кружки, постеры и худи. Для каждого товара указана цена, размер (для одежды) и количество на складе. В таблицу `customers` внесены три клиента с различными контактными данными. В таблицу `orders` добавлены четыре заказа с различными статусами, от 'новый' до 'доставлен', что позволяет проверять корректность работы запросов на фильтрацию по статусам. В таблицу `'order_items'` добавлены записи, связывающие каждый заказ с соответствующими товарами и количеством.

Тестовые данные подобраны таким образом, чтобы они позволяли проверить все функциональные возможности системы: добавление новых записей, изменение статусов заказов, формирование отчётов о популярных товарах и выручке, а также проверку остатков товаров на складе.

Проверка корректности создания базы данных

После создания базы данных и заполнения её тестовыми данными были выполнены проверочные запросы для подтверждения корректности структуры.

С помощью команды `DESCRIBE` была проверена структура каждой таблицы — типы полей, наличие первичных и внешних ключей,

ограничения NOT NULL и уникальности. Все поля соответствовали запланированным характеристикам.

С помощью запроса `SELECT * FROM` для каждой таблицы были просмотрены все добавленные записи. Данные отображались корректно, все внешние ключи указывали на существующие записи в связанных таблицах, что подтверждает правильность установленных связей.

Также была проверена работа механизма внешних ключей путём попытки добавления записи с несуществующим идентификатором клиента или товара. СУБД корректно блокировала такие операции, выводя сообщения об ошибке нарушения ссылочной целостности.

Таким образом, база данных успешно создана, структура соответствует проекту, тестовые данные загружены, все ограничения целостности работают корректно. База данных готова к выполнению аналитических запросов, которые будут рассмотрены в следующей главе.

Глава 4. Тестирование и примеры запросов

Подготовка тестовых данных

Для проверки работоспособности системы были добавлены следующие тестовые данные.

Товары:

id	Название	Категория	Цена	Размер	Остаток
1	Футболка "Рок-звезда"	Футболка	1299.00	M	25
2	Футболка "Рок-звезда"	Футболка	1299.00	L	20
3	Кружка с логотипом	Кружка	599.00	NULL	50
4	Постер "City Nights"	Постер	899.00	A3	15
5	Худи "Оверсайз"	Худи	2499.00	XL	10

Клиенты:

id	ФИО	Email	Телефон	Адрес
1	Иванов Пётр Сергеевич	ivanov@mail.ru	+79111234 567	г. Москва, ул. Тверская, д. 1
2	Смирнова Анна Викторовна	anna.smirnova@yandex.ru	+7921987 6543	г. Санкт-Петербург, Невский пр., д. 25

3	Козлов Дмитрий Алексеевич	d.kozlov@gmail.co m	+7903123 4567	г. Екатеринбург, ул. Ленина, д. 10
---	------------------------------	------------------------	------------------	---------------------------------------

Заказы (созданы с разными датами и статусами):

id	Клиент (id)	Дата заказа	Статус	Сумма	Адрес доставки
1	Иванов (1)	2025-05-10 14:30:00	доставлен	2597.00	г. Москва, ул. Тверская, д. 1
2	Иванов (1)	2025-05-15 10:20:00	отправлен	1299.00	г. Москва, ул. Тверская, д. 1
3	Смирнова (2)	2025-05-20 16:45:00	оплачен	3497.00	г. Санкт-Петербург, Невский пр., д. 25
4	Козлов (3)	2025-05-22 09:15:00	новый	1898.00	г. Екатеринбург, ул. Ленина, д. 10

Состав заказов:

id	Заказ (id)	Товар (id)	Количество	Цена на момент заказа
1	1	1	1	1299.00
2	1	3	2	599.00
3	2	1	1	1299.00
4	3	2	1	1299.00
5	3	5	1	2499.00
6	4	4	2	899.00

4.2. Проверка функциональных операций

4.2.1. Добавление товара

Действие: выбрано меню «1», введены данные: «Свитшот "Urban Style"», «Свитшот», 1899.00, "XL", 8 экземпляров.

Результат: товар появился в БД. Проверка через SELECT:

```
SELECT * FROM products WHERE name = 'Свитшот "Urban Style"';
```

id	name	category	price	size	stock_quantity	created_at
6	Свитшот "Urban Style"	Свитшот	1899.00	XL	8	2025-05-25 12:00:00

Оформление заказа

Действие: выбрано меню «3», введены customer_id = 3 (Козлов), адрес доставки, список товаров: (1, 1) — футболка в количестве 1 шт.

Результат: в таблице orders появилась новая запись с датой заказа 2025-05-25. Проверка остатков:

```
SELECT
    p.name,
    p.stock_quantity,
    SUM(oi.quantity) as заказано
FROM products p
LEFT JOIN order_items oi ON p.id = oi.product_id
LEFT JOIN orders o ON oi.order_id = o.id AND o.status IN ('новый',
'оплачен', 'отправлен')
WHERE p.id = 1
```

```
GROUP BY p.id;
```

name	stock_quantity	заказано
------	----------------	----------

Футболка "Рок-звезда"	25	2
-----------------------	----	---

Выдано 2 из 25 экземпляров, система не допустила превышения.

Изменение статуса заказа

Действие: выбрано меню «4», введен order_id = 4, новый статус = "оплачен".

Результат: поле status обновлено. Проверка:

```
SELECT id, status FROM orders WHERE id = 4;
```

id	status
4	оплачен

Отчёт о самых популярных товарах

Запрос: ТОП-3 товаров по количеству продаж.

```
SELECT
    p.name AS товар,
    p.category AS категория,
    SUM(oi.quantity) AS всего_продано
FROM order_items oi
JOIN products p ON oi.product_id = p.id
JOIN orders o ON oi.order_id = o.id
WHERE o.status IN ('оплачен', 'отправлен', 'доставлен')
GROUP BY p.id
ORDER BY всего_продано DESC
```



```
LIMIT 3;
```

Результат:

товар	категория	всего_продано
Футболка "Рок-звезда"	Футболка	2
Постер "City Nights"	Постер	2
Кружка с логотипом	Кружка	2

Отчёт о выручке по категориям

Запрос: выручка по каждой категории товаров.

```
SELECT
    p.category AS категория,
    SUM(oi.quantity * oi.price_at_order) AS выручка
FROM order_items oi
JOIN products p ON oi.product_id = p.id
JOIN orders o ON oi.order_id = o.id
WHERE o.status IN ('оплачен', 'отправлен', 'доставлен')
GROUP BY p.category
ORDER BY выручка DESC;
```

Результат:

категория	выручка
Худи	2499.00
Футболка	1299.00
Постер	898.00
Кружка	1198.00

Отчёт об активных клиентах

Запрос: клиенты с наибольшим числом заказов.

```
SELECT
    c.full_name AS клиент,
    c.email AS email,
    COUNT(o.id) AS количество_заказов,
    SUM(o.total_amount) AS сумма_покупок
FROM customers c
LEFT JOIN orders o ON c.id = o.customer_id
WHERE o.status IN ('оплачен', 'отправлен', 'доставлен')
GROUP BY c.id

ORDER BY количество_заказов DESC;
```

Результат:

клиент		email	количество_заказов	сумма_покупок
				к
Иванов Пётр Сергеевич		ivanov@mail.ru	2	3896.00
Смирнова	Анна	anna.smirnova@yandex.r	1	3497.00
Викторовна		u		

4.3.4. Дополнительный запрос: свободные остатки товаров

```
SELECT
    name AS товар,
    stock_quantity AS остаток
FROM products

ORDER BY stock_quantity DESC;
```

Результат:

товар	остаток
Кружка с логотипом	50

Футболка "Рок-звезда"	25
Худи "Оверсайз"	10
Постер "City Nights"	15

Результаты тестирования

Функция	Ожидаемый результат	Фактический результат	Статус
Добавление товара	Новая запись в products	Запись добавлена	Pass
Добавление клиента	Новая запись в customers	Запись добавлена	Pass
Оформление заказа (есть товар)	Новая запись в orders и order_items	Запись создана	Pass
Оформление заказа (нет товара)	Сообщение об ошибке	Сообщение выведено	Pass
Изменение статуса заказа	Поле status обновлено	Статус изменён	Pass
Отчёт о популярных товарах	Сортировка по убыванию	Выведен корректно	Pass
Отчёт о выручке по категориям	Сумма по категориям	Выведен корректно	Pass
Отчёт об активных клиентах	Рейтинг клиентов	Выведен корректно	Pass

Вывод по тестированию: все функциональные требования выполнены, база данных работает корректно, запросы возвращают ожидаемые

результаты. Система успешно справляется с операциями CRUD, контролирует наличие товаров и формирует аналитические отчёты.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта была разработана база данных для сайта по продаже мерча и прикладное приложение на языке Python.

Результаты работы:

1. Проведён анализ предметной области – выделены основные сущности (товары, клиенты, заказы, состав заказа), определены бизнес-процессы и функциональные требования.
2. Спроектирована база данных – построена ER-диаграмма, разработана логическая и физическая модель, выполнена нормализация до 3НФ. Связи между таблицами реализованы через внешние ключи.
3. Реализована база данных в MySQL – созданы четыре таблицы, добавлены тестовые данные (5 товаров, 3 клиента, 4 заказа).
4. Разработано приложение на Python – реализованы функции CRUD для всех сущностей, оформление заказа с контролем наличия товаров, изменение статуса заказов, консольное меню для взаимодействия.
5. Выполнено тестирование – проверены все функции, составлены и выполнены аналитические SQL-запросы (популярные товары, выручка по категориям, активные клиенты, остатки товаров). Все тесты пройдены успешно.

СПИСОК ЛИТЕРАТУРЫ

1. Дейт К. Дж. «Введение в системы баз данных». – 8-е изд. – М.: Вильямс, 2020. – 1328 с.
2. Лутц М. «Изучаем Python». – 5-е изд. – М.: Диалектика, 2019. – 1312 с.
3. Тейлор А. «SQL для чайников». – 9-е изд. – М.: Диалектика, 2019. – 416 с.
4. Ульман Д., Уидом Д. «Основы реляционных баз данных». – М.: Лори, 2018. – 384 с.
5. Гарсиа-Молина Г., Ульман Д., Уидом Д. «Системы баз данных». – М.: Вильямс, 2020. – 1088 с.
6. Официальная документация MySQL. – Режим доступа: <https://dev.mysql.com/doc/>
7. Официальная документация Python. – Режим доступа: <https://docs.python.org/3/>
8. Официальная документация mysql-connector-python. – Режим доступа: <https://dev.mysql.com/doc/connector-python/en/>

Ссылка на гит

(<https://github.com/varvarazxc/curs>)

